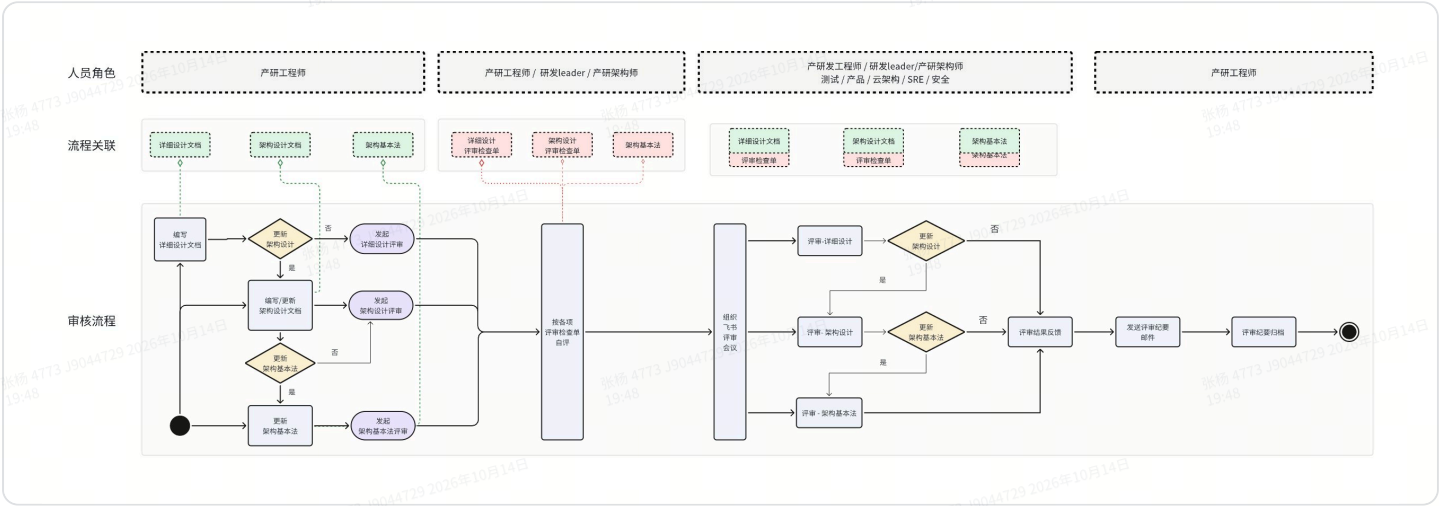
马上云-计算资源池 k8s集群按照资源池维度超分 技术设计方案-等级2

作者	版本	审核	时间	备注
江振瑜	草稿版		2024/9/15	

参考规范： MSXF-IT-RDM-GF-马上云技术设计评审规范 评审流程及材料



1. 引言

1.1 方案目的

> 这部分要描述文档的目的。应该包括需求来源、交付目的。需指明读者范围。

k8s集群按照资源池维度超分析需求功能技术详细设计方案是马上消费金融股份有限公司的需求功能详细设计技术方案文档。本文档是研发编码、功能测试、性能测试、安全测试、上线交付的基础，供马上云全体领导、项目经理、产品人员、开发人员、测试人员、运维人员、安全人员等的使用、查阅。

1.2 方案背景

> **应包含以下几个方面的内容：**
> A. 待开发软件系统名称；
> B. 该系统基本概念，如该系统的类型、从属地位等；
> C. 开发项目组名称；

> D. 技术背景、前期工作。

软件系统名称：马上云

软件模块：计算资源池

开发项目组名称：架构部-团体云团队

1.3 关联资料

> 列举本文所用的其它文档：需求文档，架构基本法，架构设计文档等。

名称	文档	描述	备注
需求产品文档	马上云需求-计算资源池-K8S资源池超分需求-等级三		产品经理：黄秋烨
架构基本法	http://devops.msxf.local/doc?url=http%3A%2F%2Fdoc.msxf.local%2Fuser%2FviewDoc%3FdocId%3D1830805758064910338		
系统架构设计	马上消费金融系统架构设计方案-马上云		
详细设计检查单	从 MSXF-IT-RDM-JCD-马上云详细设计评审检查单 copy 新建的检查单		

1.4 术语、缩略语及缩写词

> 列举本文所用的专门术语的定义和英文缩写词的原文及解释。
>
> 注意，术语之间如有引用关系，被引用者列在前面。

术语	名称	描述	备注

1.5 需求概述

> 描述本详细设计文档所描述的需求目标、可量化或可定性的价值、具体需求功能名称清单等。

需求文档：马上云需求评审-计算资源池-GPU/本地盘容量管理-等级3

需求名	价值	目标
	提升集群利用率	k8s集群按照资源池维度超分

2. 整体设计变更

> 注意：整体设计变更部分关注本次详细设计关联的架构基本法，系统架构设计变更内容。 若有对应变更，详细设计评审环节需提供架构基本法及系统架构设计文档进行变更内容讲解。

2.1 架构基本法及系统架构设计变更

> 说明是否涉及架构基本法变更，若有描述架构基本法变更内容， 例如：

本设计涉及架构基本法变更，变更内容：

1. 应用架构部分新增应用
2. 数据架构部署新增ES 数据数据存储
3. 技术架构引入新组件
4. ...

具体参见 架构基本法-马上云分册-计算

> 说明是否涉及整体架构设计变更，若有描述系统架构设计文档变更内容，例如：

本详细设计涉及系统架构设计设计变更：

1. 中间件架构变更引入ES
2. 组网部署结构调整
3. 业务连续性设计增加系统故障设计

具体参见： **马上消费金融系统架构设计方案-马上云网络VPC架构设计**

2.2 运行环境变更

> 列举本次设计涉及的基础组件变更，包含基础组件版本、配置变更，注意：仅列举变更概要描述。若涉及依赖组件类型新增变动、版本变更，**请更新架构基本法技术架构**

变更组件名	变更内容	组件变更方案
neutron	升级版本至xxxx	方案文档链接（若已评审），未评审则简要描述方案

2.3 约束限制

> 详细描述系统所受的内部和外部条件的约束和限制说明。包括业务和技术方面的条件与限制以及进度、管理等方面的限制。

约束项	描述	备注

2.4 同行产品实现初步分析

> 对本需求在阿里云、腾讯云、华为云上对应产品实现分析。

Pod分配资源压缩

代码块

```
apiVersion: v1
kind: Pod
metadata:
  name: cpu-overload
  namespace: test
spec:
  containers:
  - name: cpu-overload
    image: stress/stress:latest
    resources:
      limits:
        cpu: "2"
        memory: 1000Mi
      requests:
        cpu: "1"
        memory: 500Mi
...
```

使用调整 Pod (workload)的 request / limit 比值提升集群剩馀可分配资源。

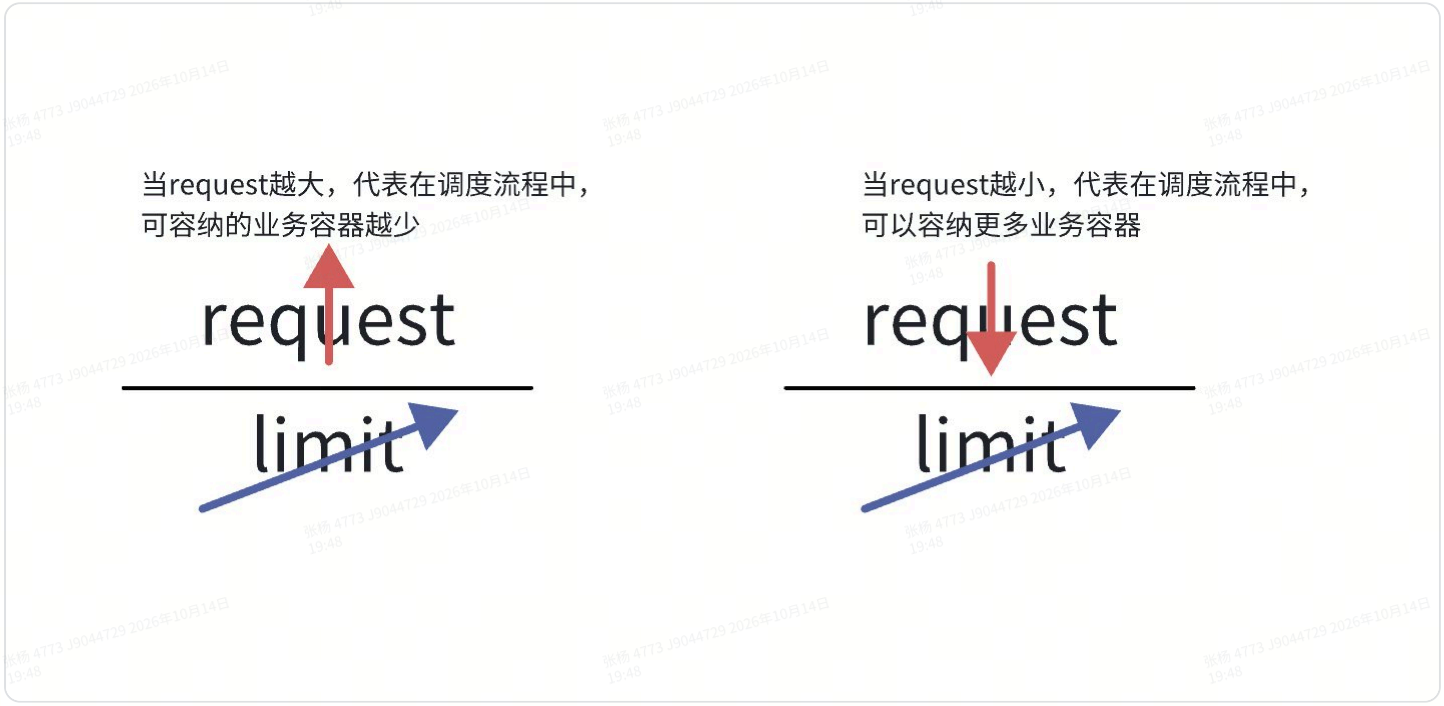
1. **Request（请求）**：
   - `request` 定义了容器启动时需要保证的最小资源量。这表示Kubernetes在调度Pod到节点时，会确保该节点上有足够的资源来满足Pod的 `request` 。只有当节点上的可分配资源大于或等于Pod的 `request` 时，Pod才会被调度到该节点上。

     Kuberentes 源码位置：

https://github.com/kubernetes/kubernetes/blob/master/pkg/scheduler/framework/types.go#L909

- `request` 不会限制容器使用的资源上限。容器在运行过程中，如果业务需求增长，可以使用超过 `request` 的资源，但最高只能使用到 `limit` 所定义的资源量。
- `request` 保证了Pod有足够的资源来运行，是容器运行时的基本保障。

2. **Limit（限制）**：
   - `limit` 定义了容器能使用的资源的最大值。如果设置为0，表示对资源使用不做限制，容器可以无限制地使用资源。
   - `limit` 的作用是防止某个Pod无限制地使用资源，导致其他Pod崩溃或影响整个集群的稳定性。
   - 在定义Pod时，必须满足 `0 <= request <= limit` 的关系。这意味着 `request` 的值总是小于或等于 `limit` 的值。



QoS（Quality of Service） 简介 QoS（Quality of Service），“服务质量等级”是作用在 Pod 上的一个配置，当 Kubernetes 创建一个 Pod 时，它就会给这个 Pod 分配一个 QoS 等级，可以是以下等级之一：

Guaranteed：Pod 里的每个容器都必须有内存/CPU 限制和请求，而且**值必须相等**。如果一个容器只指明limit而未设定request，则request的值等于limit值。

Burstable：Pod 里至少有一个容器有内存或者 CPU 请求且不满足 Guarantee 等级的要求，即**内存/CPU 的值设置的不同。**

BestEffort：容器必须没有任何内存或者 CPU 的限制或请求。

如果不设，Pod虽然很容易被调度，可以使用整个node资源，但是当资源紧俏时，会被第一个kill。

该配置不是通过一个配置项来设置，而是通过配置 CPU/MEM的 `limits` 与 `requests` 值的大小来确认 `服务质量等级` 。

可压缩资源：CPU 在压缩资源部分已经提到CPU属于可压缩资源，当pod使用超过设置的*limits*值，pod中进程使用cpu会被限制，但不会被kill。

不可压缩资源：内存 Kubernetes通过cgroup给pod设置QoS级别，当资源不足时先kill优先级低的pod，在实际使用过程中，通过OOM分数值来实现，OOM分数值从0-1000。

问题点：

- [ ] 每个Workload负载变动规律不同，因此Pod分配资源压缩比例也对应不一样，需要支持每个Workload自定义配置，而且这是对用户无感知的。
- [ ] 压缩比，设置多少合理?
- [ ] 需要搭配**重调度**方案使用会相对合理

  e.g. 比如某Workload连续7d的负载持续很低，那么改变压缩比设置，以此让集群剩余可分配资源更大，容纳更多的业务容器。

Pod 1: 2 requset limit

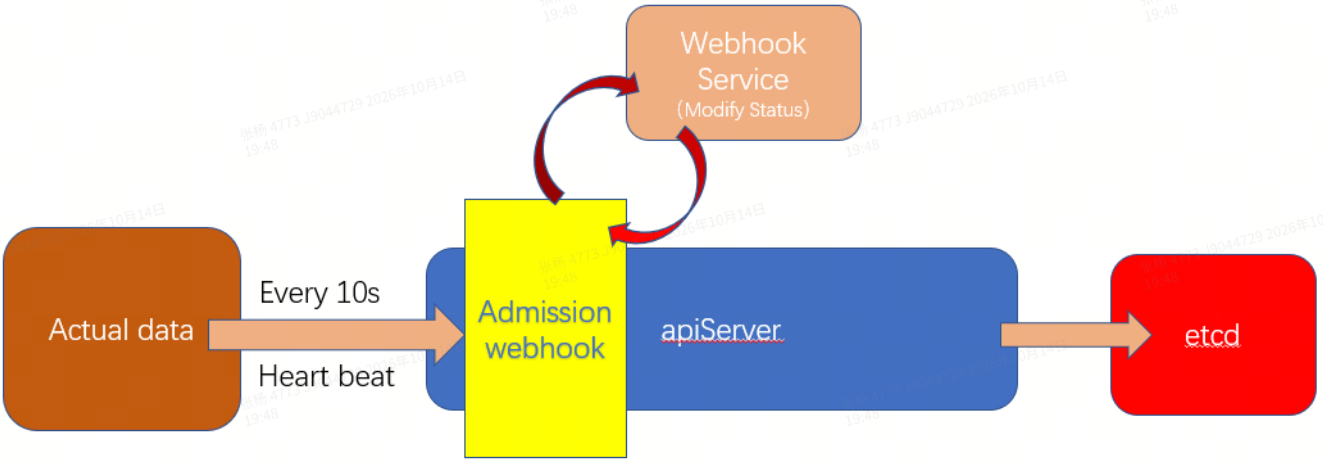
Node no

-> pod 1: 1 node 1: 2 需要**重调度**

Node资源超卖 （节点放大）。

40 -> 2* 40 ->80 api server

节点Node的Status种有两个字段，分别是**Allocatable**（可分配）和**Capacity**(总量)。"资源超卖"关键在于**动态修改**节点Node对象的allocatable字段值。如果使用clinet-go动态的修改这个字段，是无法实现的。因为修改后只是短暂的，Node资源对象比较特殊，计算节点会**不断给ApiServer发送心跳（默认每隔10s发一次），将带有Status字段的真实信息发送给ApiServer**，并更新到etcd中。也就是无论你怎么通过patch/put方法去修改Node的Status字段，计算节点都会定时通过发送心跳将真实的Status数据覆盖你修改的数据，也就是说我们无法通过直接调用RESTful API修改Node对象中的Status数据。所以这时候需要引入**MutatingWebhookConfiguration**资源对象，在apiserver发送给etcd之前，会先发送给MutatingWebhookConfiguration。然后通过我们注册的**回调函数MutateNode**，来修改status，再发送给etcd。



风险项

- [x] ~~Node status更新太频繁，每次status update都会触发webhook，大规模集群容易对apiserver造成性能问题，怎么解决？~~
  - [ ] 例如：西永集群 (node 数量 100 个节点)
  - [x] ~~Node 10s 15s 一次 100个节点~~

    每个节点上报的频率：每个节点每10秒到15秒上报一次。

    取时间范围平均值，即 (10 + 15) / 2 = 12.5 秒。

    节点数量：约100个节点

    计算每个节点平均每秒上报的次数，然后乘以节点总数来得到总的QPS。

    每个节点每12.5秒上报一次，意味着每个节点平均每秒上报的次数为 1 / 12.5 = 0.08 次。
    因此，100个节点平均每秒上报的总次数为 100 * 0.08 = 8 次。
    综上所述，100个节点在每10秒到15秒上报一次的情况下，预计产生的QPS大约为8次。

Sre: 可以在sit先运行一段时间，并确认风险与验证大规模下是否会出现类似性能问题再上线到西永环境

- [ ] 节点资源超卖对Kubelet Eviction的配置是否也有超配效果，还是仍然按照实际Node配置和负载进行evict? 如果对Evict有影响，又该如解决?

  Sre: 超卖代表更多的pod会被调度到超卖节点上，但cpu是可压缩资源，驱逐对cpu 影响不是特别大

- [ ] 超卖比例 超卖多少？是否只能由大往小调？这里是否有风险，该如何处理?

  按照产品意思最大限制是10

- [ ] 监控系统对Node的监控与Node Allocatable&Capacity Resource有关，超卖后，意味着监控系统对Node的监控不再正确，如何让监控系统也能动态的感知超卖比例进行数据和视图的修正？

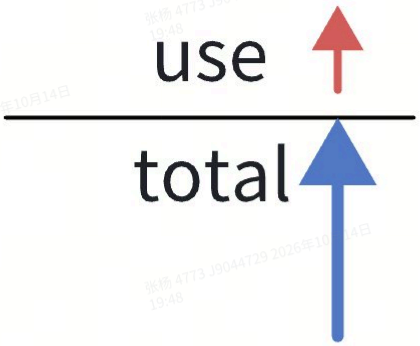
  Sre: 等正式上线后需要sre同步修改监控系统取值

- [ ] 当 webhook 停止后或超分率变小， 原本的超分节点是否会有影响？

  例如： 20核 -> 超分2 变为 40核，运行一段时间后，webhook 停掉，api-server内信息会重新变为20核。

- [ ] 对集群使用率的提升是否有限？

超分后，可容纳的pod变多，使用会增加，但是total 同样会增加



产品需求文档内容：

3.1.5 页面/功能/数据权限

无新增，沿用openstack资源池超分权限

3.1.6 数据指标/字段说明

名称	定义	约束条件	默认显示文案	备注
CPU超分率	用于控制分配给云主机的虚拟CPU数量。计算公式： （物理CPU总cpu-预留CPU） X CPU超分率 = 可分配的虚拟CPU数量。	小数位支持保持两位 超分最大数量为10	-	
内存超分率	用于控制分配给云主机的虚拟CPU数量。计算公式： （物理内存 -内存预留）X 内存超分率 = 可分配的虚拟内存数量。	小数位支持保持两位 超分最大数量为10	-	
CPU limit	物理机节点上所有pod限制的CPU总量	单位核数		

另外，产品有设置场景分析，是否可以优先在测试环境运行一段时间，等证明在测试环境有用，并没有风险或危害，再上线到正式环境

研发侧评估：资源压缩与节点放大方案**不并存**。

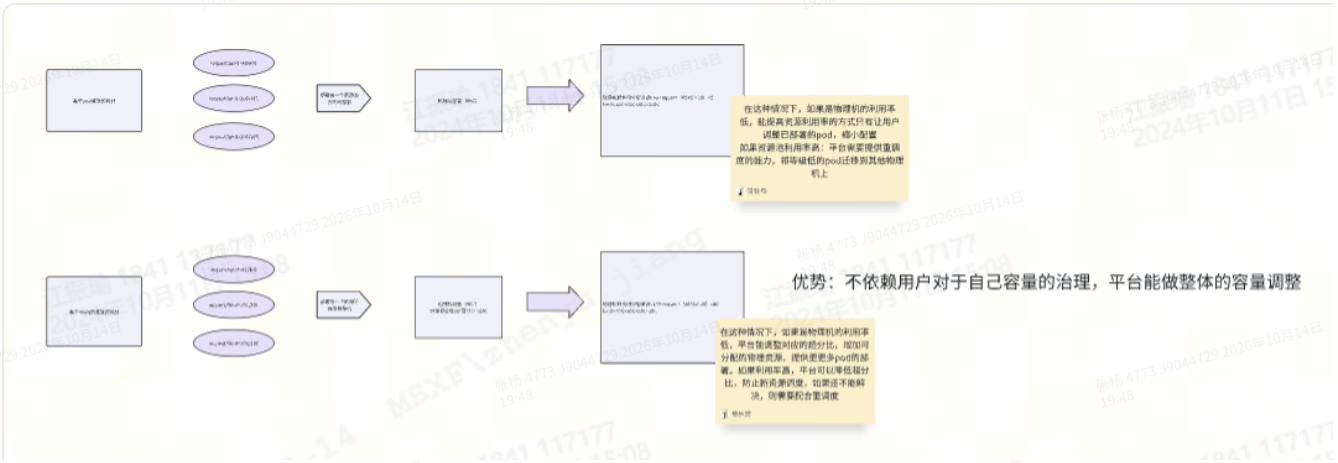
```
例如：
上线西永环境：
1．先运行一段时间后，
```

场景分析：

1、**针对马消租户，西永测试环境（可以同时开启pod维度超分和node超分）**

2、**针对生产环境，针对马消租户独享的资源池，可根据具体的业务，评估使用node超分还是pod超分**

3、**针对公有云租户，建议使用node超分，所有容器的部署，request=limit**



2.5 安全设计

> 根据云安审等研发过程要求，安全设计贯穿各个设计环节，对于详细设计阶段也需评估进行安全设计。
>
> **若有涉及安全设计点，务必提前邀请安全技术团队（信息科技安全技术团队-应用与业务逻辑安全组，负责人：黄荣存）参与评审**

2.5.1 物理安全

> 【说明：服务器部署物理环境安全，采用何种方式确保承载业务的服务器物理安全，防火、防水、电力、温湿度、门禁防盗窃等方面。如xx系统服务器采用xx云服务器/托管xx机房，物理环境安全满足ISO27001/等保的相关机房安全要求。】

2.5.2 网络安全

> 【说明：描述业务系统网络区域规划、隔离、内外部访问控制方案。简要绘制拓扑图，呈现网络访问控制、冗余高可用设计、业务流程关系等。】
>
> **特别的：**
>
> 1. **云上云下访问链路**
> 2. **安全组规则设计（若涉及）**

2.5.3 主机安全

> 【说明：如xx系统服务器操作系统，采用统一xx主机安全基线配置加固，是否具备主机资源监控、恶意代码防范、入侵检测发现及预警能力。】

2.5.4 应用安全

> 【说明：描述应用系统用户角色、权限、身份认证、访问控制，应用操作日志与审计功能设计方案。如何处理、限制异常登录，包括但不限于暴力破解、撞库等攻击行为。描述采用何种安全开发框架、方案规避常见Web安全漏洞，如SQL注入、XSS、越权、敏感信息泄露等。是否采用Web应用安全防护（软/硬WAF）能力。】

2.5.5 数据安全

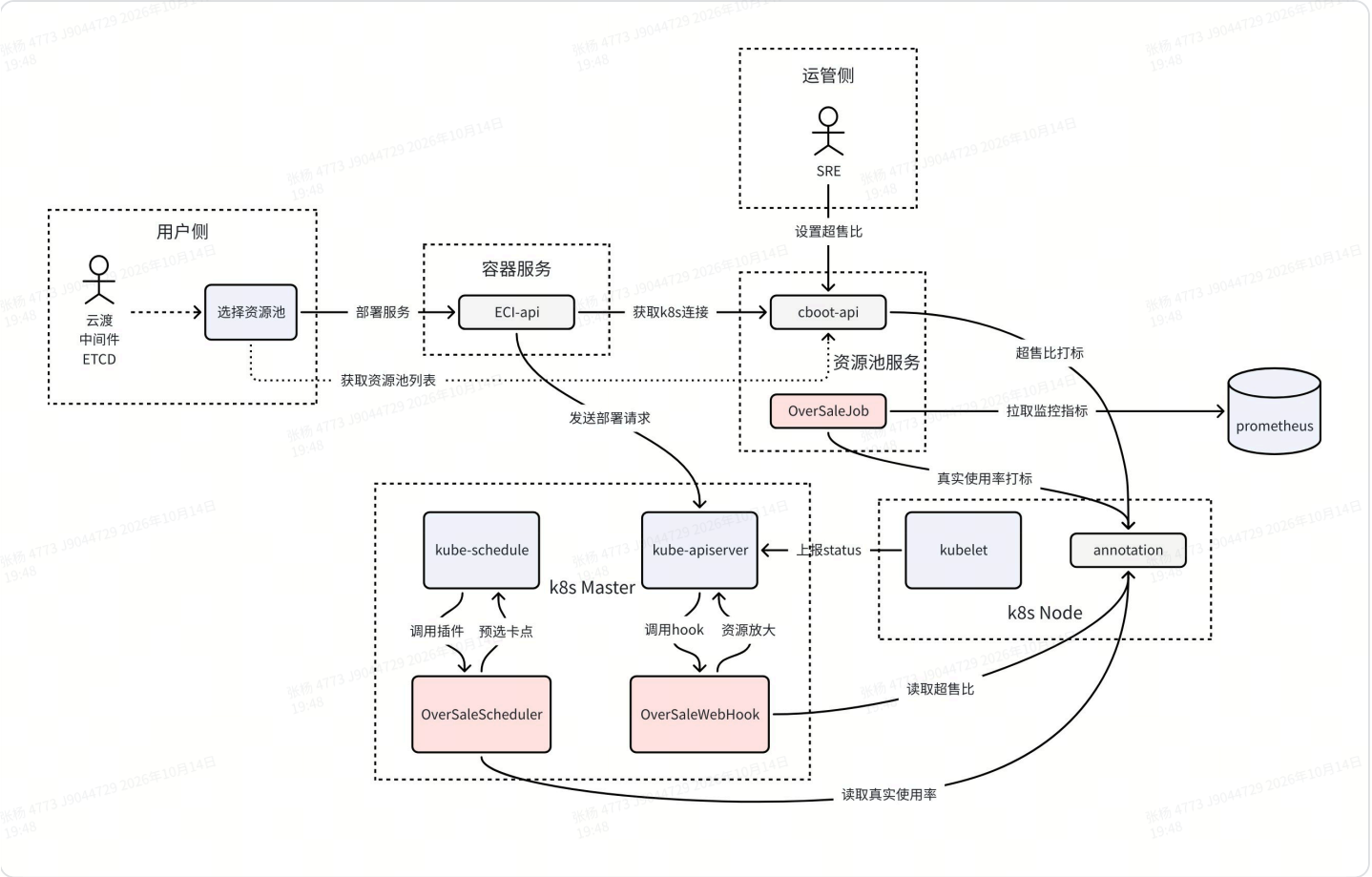
> 【说明：描述敏感数据传输加密、存储加密设计方案，数据备份与恢复设计方案】

3. 模块设计

3.1 模块划分

> 根据系统设计划分模块，并阐述模块划分的理由。可以使用图表表示。

新增的模块为**红色部分**



3.2 模块一 OverSaleJob

3.2.1 模块描述

> 描述模块的职责。

计算资源池服务中实现cron(每10分钟执行一次)，周期性对超卖资源池中的各node计算真实cpu利用率。将计算结果通过annotation注入到k8s node信息中。

加入：一个定时任务：一个小时执行一次，检查超卖资源池中节点的使用率，如果节点的利用率过期，触发告警。

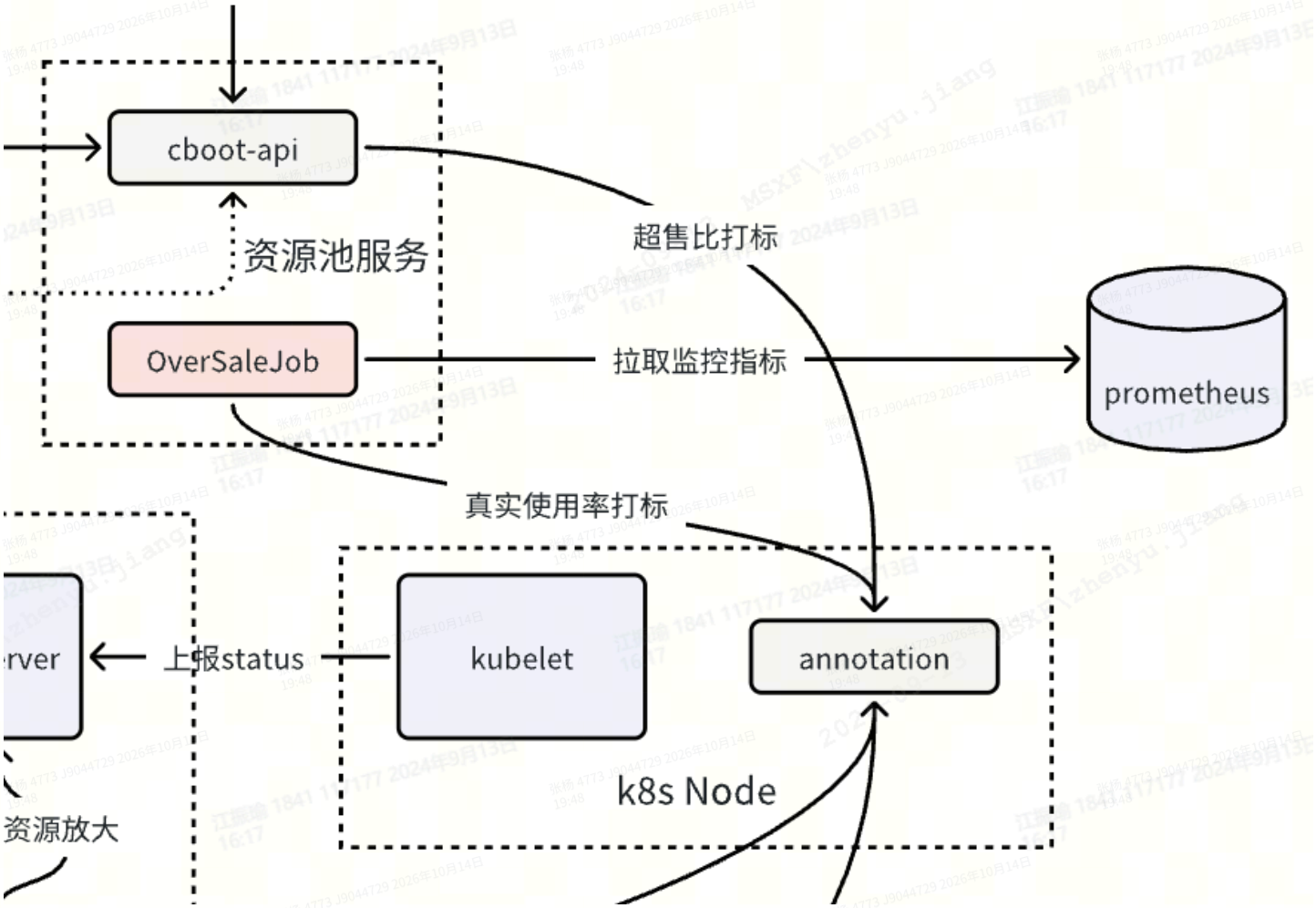
代码块

```
+-----------------------------------+
|                                   |
|  定时任务1：计算并打标CPU利用率    |
|  (每10分钟执行一次)                |
|                                   |
+----------------+------------------+
                 |
                 v
+----------------+------------------+
|                                   |
|  从Prometheus获取CPU使用数据       |
```

```
|  计算每个节点的CPU利用率           |
|  更新Kubernetes Node注解          |
|                                   |
+-----------------+-----------------+
                  |
                  v
+-----------------+-----------------+
|                                   |
|  定时任务2：检查注解并告警         |
|  （每小时执行一次）                |
|                                   |
+-----------------+-----------------+
                  |
                  v
+-----------------+-----------------+
|                                   |
|  检查每个Node的注解                |
|  如果没有找到注解，触发告警        |
|                                   |
+-----------------------------------+
```

3.2.2 模块业务流程

> 描述模块的业务流程，可以通过流程图加以说明。

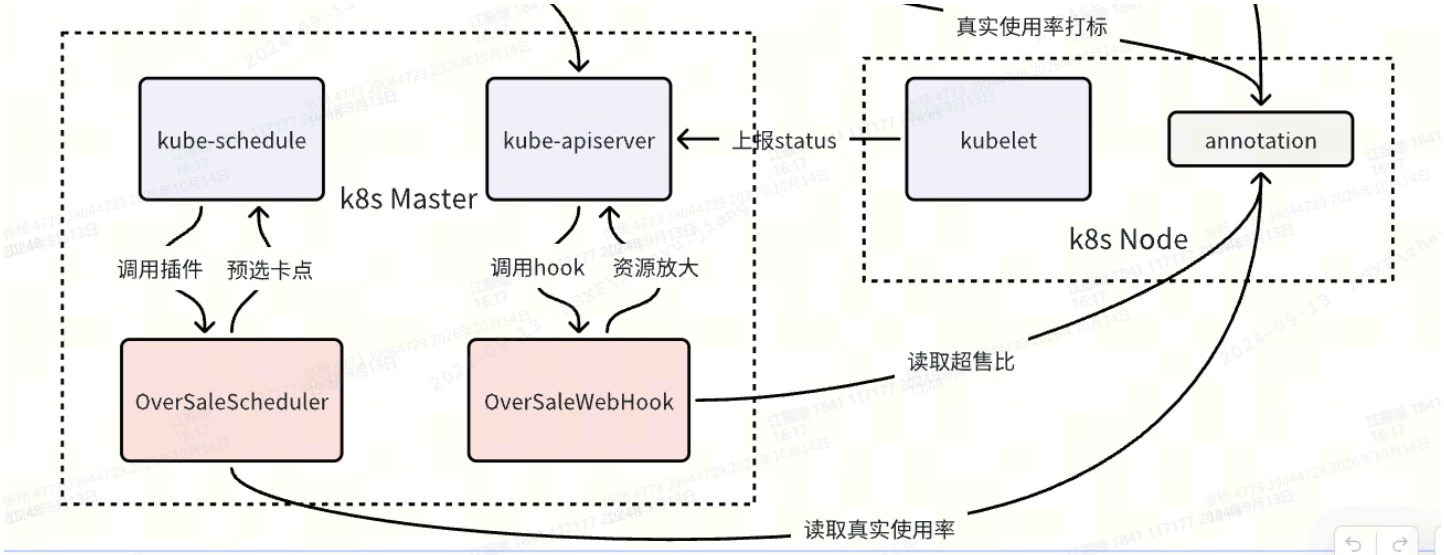


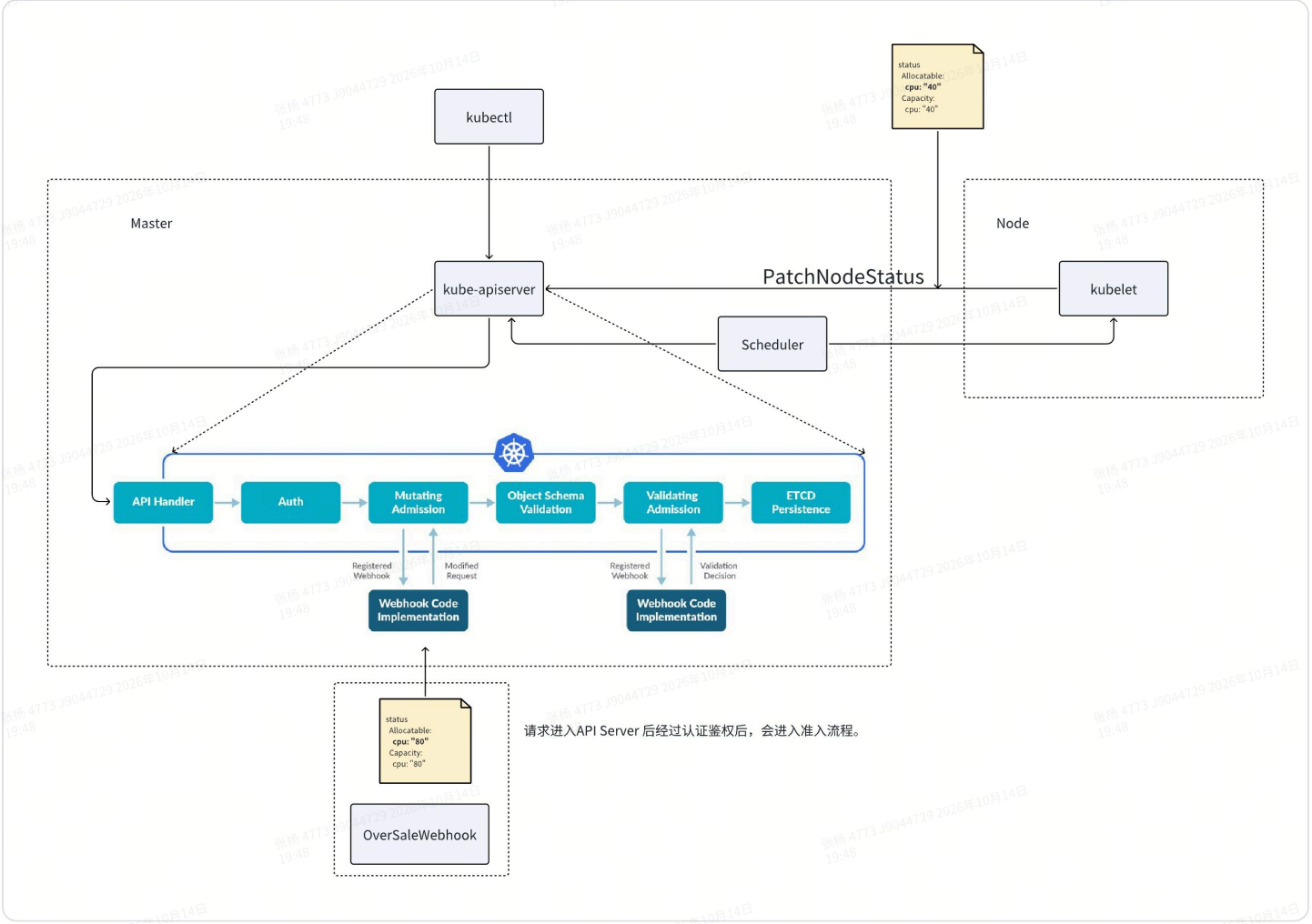
3.3 模块二 OverSaleWebHookServer

3.3.1 模块描述

部署在k8s集群master节点的webhook服务，使用k8s apiServer的mutatingAdmission机制，实现对node可调度资源的修改。

3.3.2 模块业务流程





部署与注册流程

1. 基础组件变更：资源池超卖-基础组件变更
2. 证书注册-secret.yaml

> 作为管理组件，要部署到master节点，采用statefulset部署，注册的webhook回调地址使用master vip
>
> 3 master node ip
>
> k8s master worker node ip

代码块

```
apiVersion: v1
data:
  tls.crt: {OVERSALE_TLS_CRT}
  tls.key: {OVERSALE_TLS_KEY}
kind: Secret
metadata:
  name: oversale-hook-demo-certs
```

```
  namespace: kube-system
type: kubernetes.io/tls
```

3. hook服务部署-app.yaml

代码块

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: oversale-hook-demo
  namespace: kube-system
  labels:
    app: oversale-hook-demo
spec:
  replicas: 2
  selector:
    matchLabels:
      app: oversale-hook-demo
  template:
    metadata:
      labels:
        app: oversale-hook-demo
    spec:
      hostNetwork: true
      containers:
        - name: oversale-hook-demo
          image: {OVERSALE_WEBHOOK_IMAGE}
          env:
            name: OVERSALE_CPU_RATIO
            value: "2.0"
          imagePullPolicy: Always
          securityContext:
            privileged: true
            runAsUser: 0
          args:
            - --tls-cert-file=/etc/kubernetes/pki/oversale/tls.crt
            - --tls-private-key-file=/etc/kubernetes/pki/oversale/tls.key
            - --secure-port=30802
          ports:
            - name: https
              containerPort: 30802
              protocol: TCP
          #readinessProbe:
          #  periodSeconds: 15
          #  httpGet:
```

```
            #    path: /healthz
            #    port: metrics
            volumeMounts:
              - name: webhook-certs
                mountPath: /etc/kubernetes/pki/oversale
                readOnly: true
          volumes:
            - name: webhook-certs
              secret:
                defaultMode: 420
                secretName: oversale-hook-demo-certs
```

4. hook服务注册-webhook.yaml

代码块

```
apiVersion: admissionregistration.k8s.io/v1
kind: MutatingWebhookConfiguration
metadata:
  name: oversale-hook-webhook
  labels:
    app: oversale-hook-webhook
    kind: mutator
webhooks:
  - name: oversale-hook-demo.mscloud.dev
    admissionReviewVersions: ["v1beta1"]
    sideEffects: None

    clientConfig:
      caBundle: {OVERSALE_TLS_CRT}
      url: https://{OVERSALE_MASTER_VIP}:30802/node
    failurePolicy: Igore
    matchPolicy: Exact
    namespaceSelector: {}
    objectSelector: {}
    reinvocationPolicy: Never
    timeoutSeconds: 10
    rules:
      - operations: ["UPDATE"]
        apiGroups: ["*"]
        apiVersions: ["*"]
        resources: ["nodes/status"]
        scope: '*'
```

💡 failurePolicy: 设置为Ignore，则当Webhook调用失败时，Kubernetes将继续进行操作。

代表：10C的节点上报，正常经过webhook的时候会变为上报 20c，但是如果遇到webhook挂了，

https://zhuanlan.zhihu.com/p/669490503

生命周期

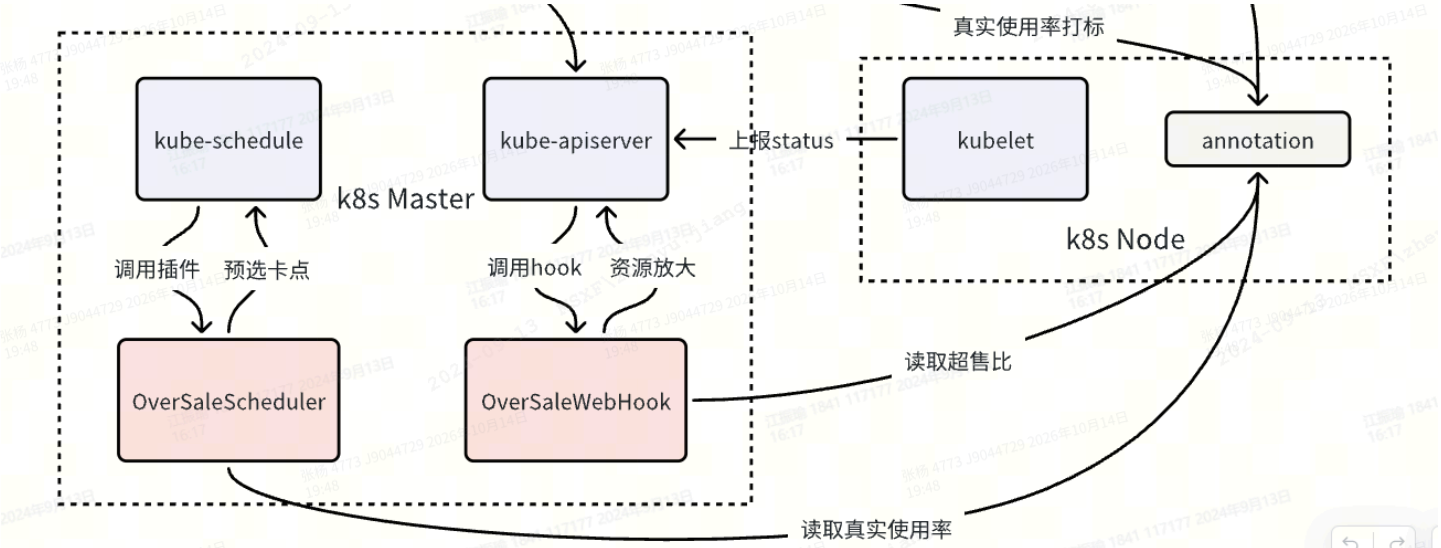
由SRE在k8s物理集群初始化后，进行基础组件安装，走基础组件变更流程。详见：资源池超卖-基础组件变更

3.4 模块三 OverSaleSchedulerPlugin

3.4.1 模块描述

部署在k8s集群master节点的调度器插件，实现预选策略对节点进行过载保护。

3.4.2 模块业务流程



部署与注册流程

1. 调度插件部署

代码块

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    component: kube-scheduler
    tier: control-plane
  name: kube-scheduler
  namespace: kube-system
spec:
```

```
11    containers:
12      - command:
13          - kube-scheduler
14          - --authentication-kubeconfig=/etc/kubernetes/scheduler.conf
15          - --authorization-kubeconfig=/etc/kubernetes/scheduler.conf
16          - --bind-address=0.0.0.0
17          - --kubeconfig=/etc/kubernetes/scheduler.conf
18          - --leader-elect=true
19          - --config=/etc/kubernetes/oversale-sched-cc.yaml
20          - --v=5
21        #image: harborxy.msxfcloud.test/base/kube-scheduler:v1.24.1
22        image: www.harbor-test1002.com/mscloud/kube-scheduler:v1.0.5
23        imagePullPolicy: IfNotPresent
24        livenessProbe:
25          failureThreshold: 8
26          httpGet:
27            host: 127.0.0.1
28            path: /healthz
29            port: 10259
30            scheme: HTTPS
31          initialDelaySeconds: 10
32          periodSeconds: 10
33          timeoutSeconds: 15
34        name: kube-scheduler-ext
35        resources:
36          requests:
37            cpu: 100m
38        startupProbe:
39          failureThreshold: 24
40          httpGet:
41            host: 127.0.0.1
42            path: /healthz
43            port: 10259
44            scheme: HTTPS
45          initialDelaySeconds: 10
46          periodSeconds: 10
47          timeoutSeconds: 15
48        volumeMounts:
49          - mountPath: /etc/kubernetes
50            name: kubeconfig
51    hostNetwork: true
52    priorityClassName: system-node-critical
53    securityContext:
54      seccompProfile:
55        type: RuntimeDefault
56    volumes:
57      - hostPath:
```

```
58            path: /etc/kubernetes/
59            type: Directory
60          name: kubeconfig
61    status: {}
```

2. 自定义调度配置文件

https://kubernetes.io/zh-cn/docs/reference/scheduling/config/#multiple-profiles

代码块

```yaml
apiVersion: kubescheduler.config.k8s.io/v1beta2
kind: KubeSchedulerConfiguration
leaderElection:
  leaderElect: true
clientConnection:
  # 连接的 kubeconfig 目录
  kubeconfig: /etc/kubernetes/scheduler.conf
profiles:
  - schedulerName: default-scheduler
    plugins:
      score:
        enabled:
          - name: "Oversale"
            # 评分权重
            weight: 1
    pluginConfig:
      # 配置文件入参
      - name: Oversale
        args:
          cpuUsageThreshold: 0.5
          cpuEstimateThreshold: 0.9
          expireHour: 2
```

生命周期

由SRE在k8s物理集群初始化后，进行基础组件安装，走基础组件变更流程。详见：资源池超卖-基础组件变更

4. 接口设计

> **各团队接口文档会汇总至各团队的总接口文档（方便内部团队间同步及外部审核要求）并持续维护**

4.1 外部依赖接口(不涉及)

> 本详细方案中，需要调用其他系统、应用的接口。

4.1.1 外部接口名称

4.1.1.1 接口定义

> 表格形式列出接口定义，包括接口名，参数，返回值，错误码等。

接口	/portal/gcloudmw-logs/beat/install		接口类型	新增接口
名称	安装filebeat		请求方法	POST
提供方	gc-mw-logs		调用方	gc-waf-portal
入参列表				
序号	**字段**	**描述**	**数据类型**	**是否必须**	**备注**
1	ips	需要安装Filebeat的实例IP	List<String>	Y	如果接入了全局管控面VPC,则是全局管控面网卡IP,List最大长度为50
	azoneCode	地域可用区编码	String	Y	地域可用区编码，例如10101261
3	resourceCode	集群资源码	String	Y	集群资源码（全局唯一）
4	vpcId	VPC资源ID	String	Y	vpc主键id(从SLB接口中取)
5	tags		List<Tag>	N	WAF没有Tag运用，暂不传
5.1	key		String	N	
5.2	value		String	N	
出参列表	**data.list**			
序号	**字段**	**描述**	**数据类型**	**备注**
1	address	节点ID列表	List<String>	
2	vpcids	VPC ID列表	List<String>	

3	grids	Grid列表	List<String>	
错误码				
1	code	0：成功	String	0以外的均为异常返回，结合msg错误信息处理
2	msg	"请求成功"	String	
3	data	出参列表	Object	
4	requestId	请求唯一ID	String	

4.1.1.2 接口流程

> 列出对此接口的处理流程图。

4.2 内部接口

> 本详细方案中，提供给**前端或内部应用调用**的接口。

4.2.1 接口名称

4.2.1.1 接口定义

> 表格形式列出接口定义，包括接口名，参数，返回值，错误码等。

资源池 k8s 超分率

设置接口

接口路径：/admin/cboot/pool/setRate

请求方式：POST

- 请求参数

代码块

```
{
    "id" : 1,                 # poolId
    "cpuRate": 3.22,          # cpu超分率
    "memRate": 1,             # mem超分率
}
```

- 接口响应

代码块

```
{
    "code": 0,
    "msg": "success",
    "requestId": "",
    "data": {}
}
```

获取接口

接口路径：/admin/cboot/pool/getRate?id=12

请求方式：GET

- 请求参数

代码块

参数	位置	类型	是否必须	描述
id	query	string	是	资源池id

- 接口响应





代码块

```
{
    "code": 0,
    "msg": "success",
    "requestId": "",
```

```
    "data": {
        "cpuRate": 3.32,        # cpu超分率
        "memRate": 1,           # mem超分率
        "cpuLoad": 2.4          # CPU 负载
    }
}
```

接口路径：/admin/cboot/node/monitor

请求方式：POST

- 请求参数

代码块

```
{
    "idList": [131]
}
```

代码块

```
{
    "code": 0,
    "msg": "success",
    "requestId": "",
    "data": {
        "id": 101,
        "cpuInuse": 0,
        "cpuTotal": 40,
        "cpuUseRate": 0,
        "cpuAllocRate": 4250,  # 分配率 万分比 也就是给 4250 -> 代表 42.50%
        "memTotal": 0,
        "memInuse": 0,
        "memUseRate": 0,
        "memAlloc": 0,
        "memeAllocRate": 2687,  # 分配率 万分比 也就是给 2687-> 代表 26.87%
        "diskTotal": 0,
        "diskAlloc": 0,
        "diskAllocRate": 10000,      # 分配率 万分比 也就是给 10000-> 代表 100%
        node pod request limit
        k8s node pods requset limit -> db cboot
        "cpuRequset": 100,
        "cpuLimit": 100,
        "memRequest": 200,
        "memLimit": 300

    }
```

```
27    }
```

4.2.1.2 核心参数补充说明

> 要求核心参数说明传递方式，取值来源。 例如ECS 创建接口 镜像ID， 镜像ID 如何确定

4.2.1.3 接口SLO要求及熔断限流评估

> 参见： 马上云应用SLO定义

类别	阈值	说明
请求成功率	99.99%	**必须定义**
TP99 延迟	5ms	**必须定义**
TP90 延迟	10ms	**必须定义**

> 根据熔断限流平台接入要求，评估熔断限流配置

限流配置：**必须定义**

熔断配置：**必须定义**

4.2.1.4 接口流程

> 列出对此接口的处理流程图。

4.2.1.5 接口幂等性设计

> 接口幂等性设计考虑，描述如何保证幂等

4.2.1.6 接口上下游资源关联参数设计

> 描述接口如何关联上游资源ID，环境ID（租户ID，订单ID 等），数据存储是否包含了关联字段。

4.2.1.7 接口兼容性设计

> 1. 明确接口实现对已有逻辑的兼容性设计考虑点
> 2. 评估业务实现对历史数据是否兼容

4.3 暴露接口(不涉及)

> 本详细方案中，提供给外部系统、应用调用的接口。

4.3.1 接口名称

4.3.1.1 接口定义

> 表格形式列出接口定义，包括接口名，参数，返回值，错误码等。

4.3.1.2 核心参数补充说明

> 要求核心参数说明传递方式，取值来源。 例如ECS 创建接口 镜像ID， 镜像ID 如何确定

4.3.1.3 接口SLO要求及熔断限流评估

> 参见： 马上云应用SLO定义

类别	阈值	说明
请求成功率	99.99%	**必须定义**
TP99 延迟	5ms	**必须定义**
TP90 延迟	10ms	**必须定义**

> 根据熔断限流平台接入要求，评估熔断限流配置

限流配置：**必须定义**

熔断配置：**必须定义**

4.3.1.4 接口流程

> 列出对此接口的处理流程图。

4.3.1.5 接口幂等性设计

> 接口幂等性设计考虑，描述如何保证幂等

4.3.1.6 接口上下游资源关联参数设计

> 描述接口如何关联上游资源ID，环境ID（租户ID，订单ID 等），数据存储是否包含了关联字段。

4.3.1.7 接口兼容性设计

> 1. 明确接口实现对已有逻辑的兼容性设计考虑点
> 2. 评估业务实现对历史数据是否兼容

5. 功能设计

5.1 功能一

5.1.1 该功能使用人群特点分析

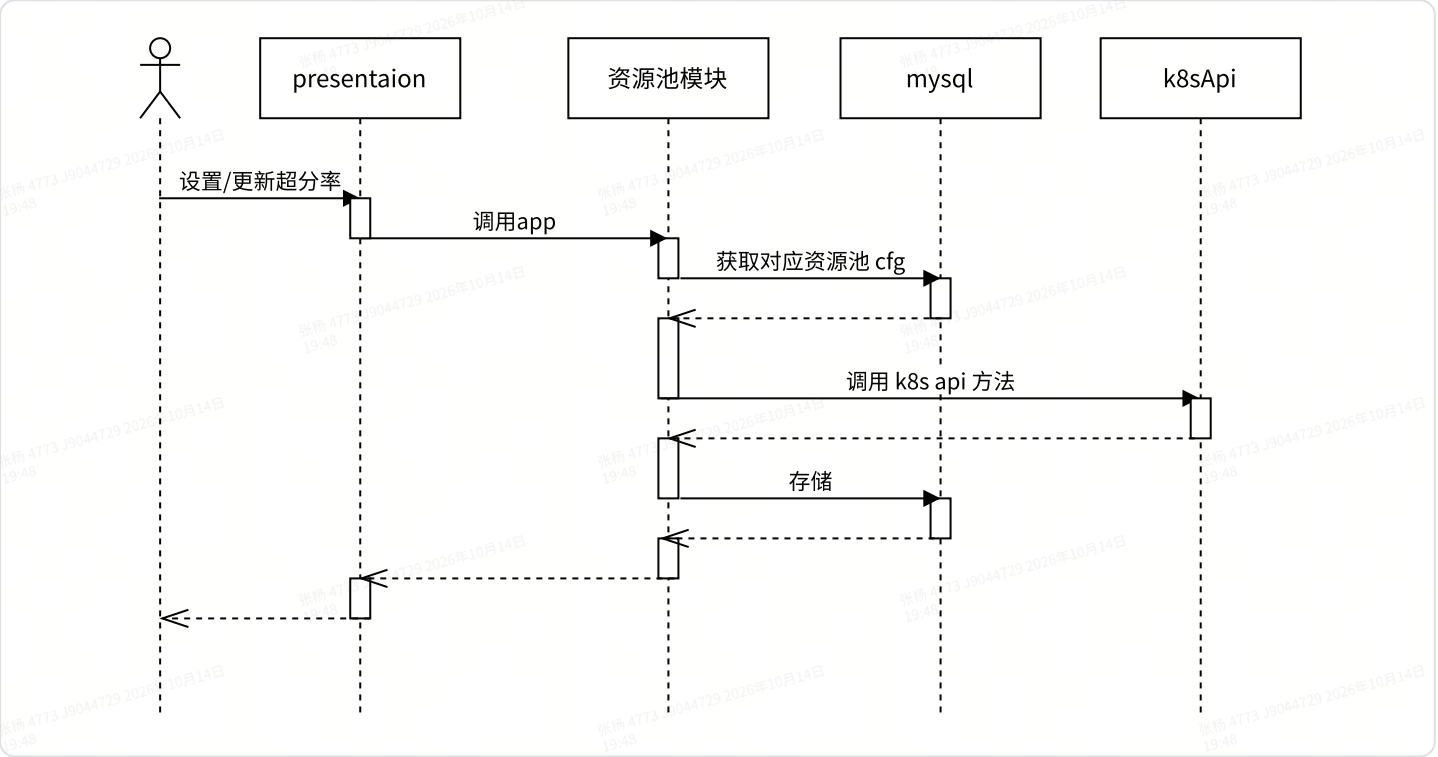
> *说明此功能的使用受众人群，使用角色范围：租户、后管或其它特殊考虑的特点*

5.1.2 功能逻辑实现设计描述

> 1. *用流程图、功能交互图、时序图、状态机等，加必要的说明来表示本功能的逻辑流程，逻辑处理核心要点*
> 2. ***评估对已有业务流程兼容性***

1. 设置/更新超分率

调用 k8s api 修改或新增node label 标签超分率信息



OverSaleJob

cpu真实使用率计算规则

http://prometheus.bn01.mscloud.local/rules#recording-kubernetes.rules

计算规则（参考了腾讯开源crane-scheduler对真实cpu使用率的算法）：

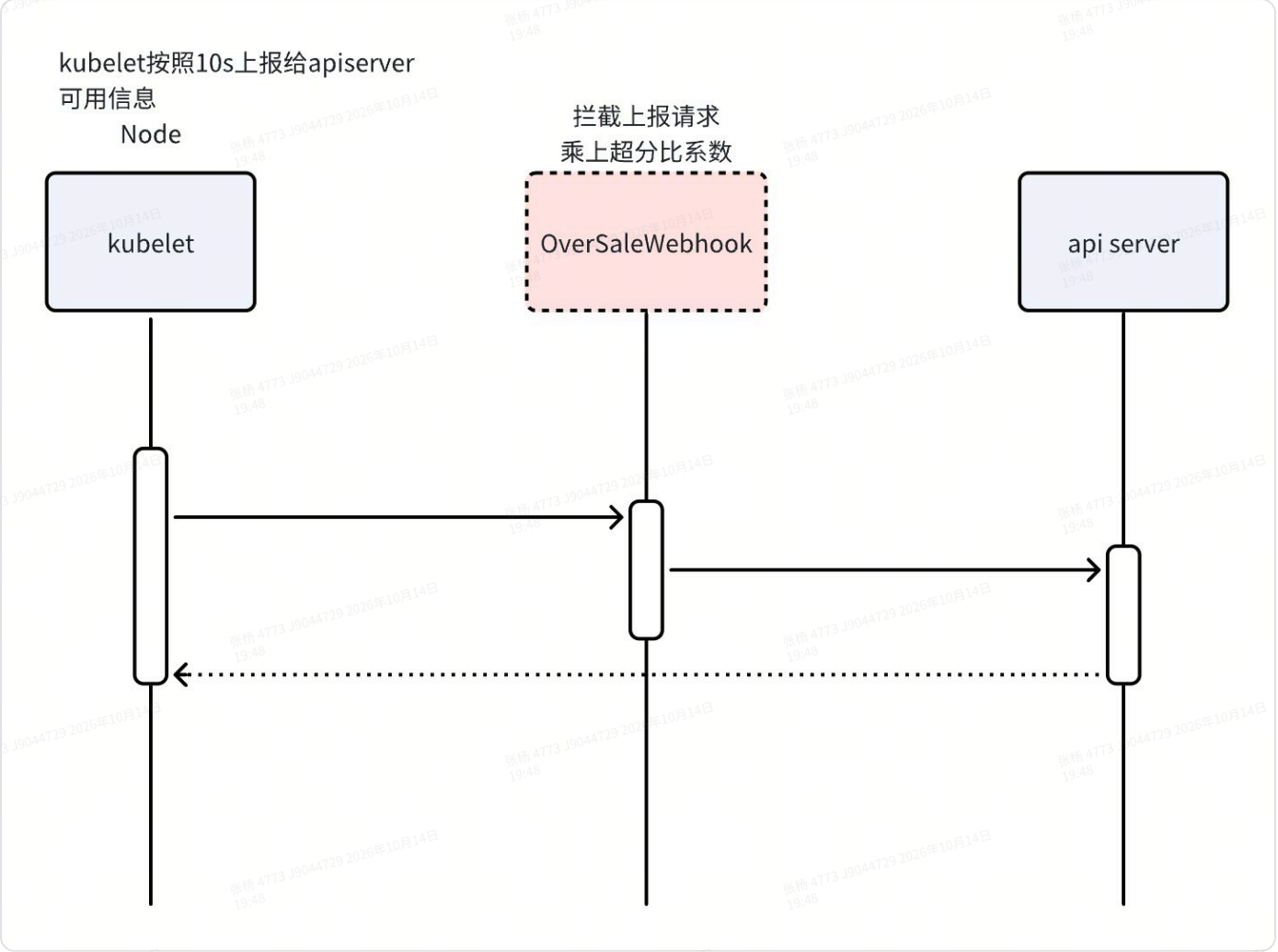
- 近5分钟的**均值cpu_usage_avg_5m** = 100 - (avg by (instance) (avg_over_time(irate(node_cpu_seconds_total{mode="idle",instance="~~10.50.1.5:9100~~"}[1m])[5m:]))) * 100
- 近1小时的**均值cpu_usage_avg_1h** = 100 - (avg by (instance) (max_over_time(irate(node_cpu_seconds_total{mode="idle",instance="~~10.50.1.5:9100~~"}[5m])[1h:]))) * 100
- 近1天的**均值cpu_usage_avg_1d** =100 - (avg by (instance) (max_over_time(irate(node_cpu_seconds_total{mode="idle",instance="~~10.50.1.5:9100~~"}[5m])[1d:]))) * 100

真实cpu利用率计算公式 = 0.2*cpu_usage_avg_5m + 0.5*cpu_usage_avg_1h + 0.3*cpu_usage_avg_1d （权重可配置）

将计算结果以annotation注入到node信息中，格式如：

- **mscloud.msxf.lo/oversale_cpu_usage_active**: 0.1512_1711231230
  - 0.1512 表示当前计算的cpu真实利用率是：15.12%(保留小数点后2位)
  - 1711231230是当次计算完成时的时间戳。用于做过期校验和超时异常告警

webhook执行流程



- 从当前node的annotation中获取超售比，如果存在且大于1.0，那么将nodeStatus中的Allocatable.cpu和Capacity.cpu乘以超售比，为确原数据，会提供 mscloud.msxf.lo/original_allocatable_cpu mscloud.msxf.lo/original_capacity_cpu 保留原cpu数据

代码块

```
1   Name:               k8s-master
2   Roles:              control-plane
```

```
Labels:             bcbootFaultDomain=DCXN-DSJFR03-0-change1
                    beta.kubernetes.io/arch=amd64
                    beta.kubernetes.io/os=linux
                    cbootFaultDomain=DCXN-DSJFR03-0-change
                    kubernetes.io/arch=amd64
                    kubernetes.io/hostname=k8s-master
                    kubernetes.io/os=linux
                    node-role.kubernetes.io/control-plane=
                    node.kubernetes.io/exclude-from-external-load-balancers=
                    poolCode=ecigroup-pool
                    promthanos=true
Annotations:        csi.volume.kubernetes.io/nodeid: {"egp-csi-nas":"k8s-master","nfs.csi.k8s.io":"k8s-master"}
                    mscloud.msxf.lo/original_allocatable_cpu: 40
                    mscloud.msxf.lo/original_capacity_cpu: 40
                    xxxx: 3
                    kubeadm.alpha.kubernetes.io/cri-socket: unix:///var/run/containerd/containerd.sock
                    node.alpha.kubernetes.io/ttl: 0
                    volumes.kubernetes.io/controller-managed-attach-detach: true
CreationTimestamp:  Tue, 15 Nov 2022 17:49:31 +0800
Taints:             node.kubernetes.io/unschedulable:NoSchedule
Unschedulable:      true
Lease:
  HolderIdentity:  k8s-master
  AcquireTime:     <unset>
  RenewTime:       Mon, 14 Oct 2024 17:11:00 +0800
Conditions:
  Type                 Status  LastHeartbeatTime                 LastTransitionTime                Reason                       Message
  ----                 ------  -----------------                 ------------------                ------                       -------
  KernelDeadlock       False   Mon, 14 Oct 2024 17:10:05 +0800   Fri, 13 Sep 2024 17:24:39 +0800   KernelHasNoDeadlock          kernel has no deadlock
  ReadonlyFilesystem   False   Mon, 14 Oct 2024 17:10:05 +0800   Fri, 13 Sep 2024 17:24:39 +0800   FilesystemIsNotReadOnly      Filesystem is not read-only
  MemoryPressure       False   Mon, 14 Oct 2024 17:11:06 +0800   Thu, 12 Sep 2024 13:52:19 +0800   KubeletHasSufficientMemory   kubelet has sufficient memory available
  DiskPressure         False   Mon, 14 Oct 2024 17:11:06 +0800   Thu, 12 Sep 2024 13:52:19 +0800   KubeletHasNoDiskPressure     kubelet has no disk pressure
```

```
  PIDPressure           False   Mon, 14 Oct 2024 17:11:06 +0800   Thu, 12 Sep 2024 13:52:19 +0800   KubeletHasSufficientPID      kubelet has sufficient PID available
  Ready                 True    Mon, 14 Oct 2024 17:11:06 +0800   Thu, 12 Sep 2024 13:52:19 +0800   KubeletReady                 kubelet is posting ready status
Addresses:
  InternalIP:  10.97.16.2
  Hostname:    k8s-master
Capacity:
  cpu:                          120
  ephemeral-storage:            575514052Ki
  hugepages-1Gi:                0
  hugepages-2Mi:                0
  kubernetes.io/batch-cpu:      22742
  kubernetes.io/batch-memory:   29626237362
  memory:                       131645984Ki
  pods:                         220
Allocatable:
  cpu:                          120
  ephemeral-storage:            530393749446
  hugepages-1Gi:                0
  hugepages-2Mi:                0
  kubernetes.io/batch-cpu:      22742
  kubernetes.io/batch-memory:   29626237362
  memory:                       131543584Ki
  pods:                         220


```

- 需要保证hook服务的性能与可用性：
  - 保持该hook服务的逻辑简单：避免io操作，只执行简单的计算

示例代码如下：

代码块

```
func (na *WebhookNodeAllocatableMutator) MutateNode(node *core.Node, admissionRequest *admissionv1beta1.AdmissionRequest) error {
        if admissionv1beta1.Update != admissionRequest.Operation || admissionRequest.SubResource != "status" {
                return nil
        }

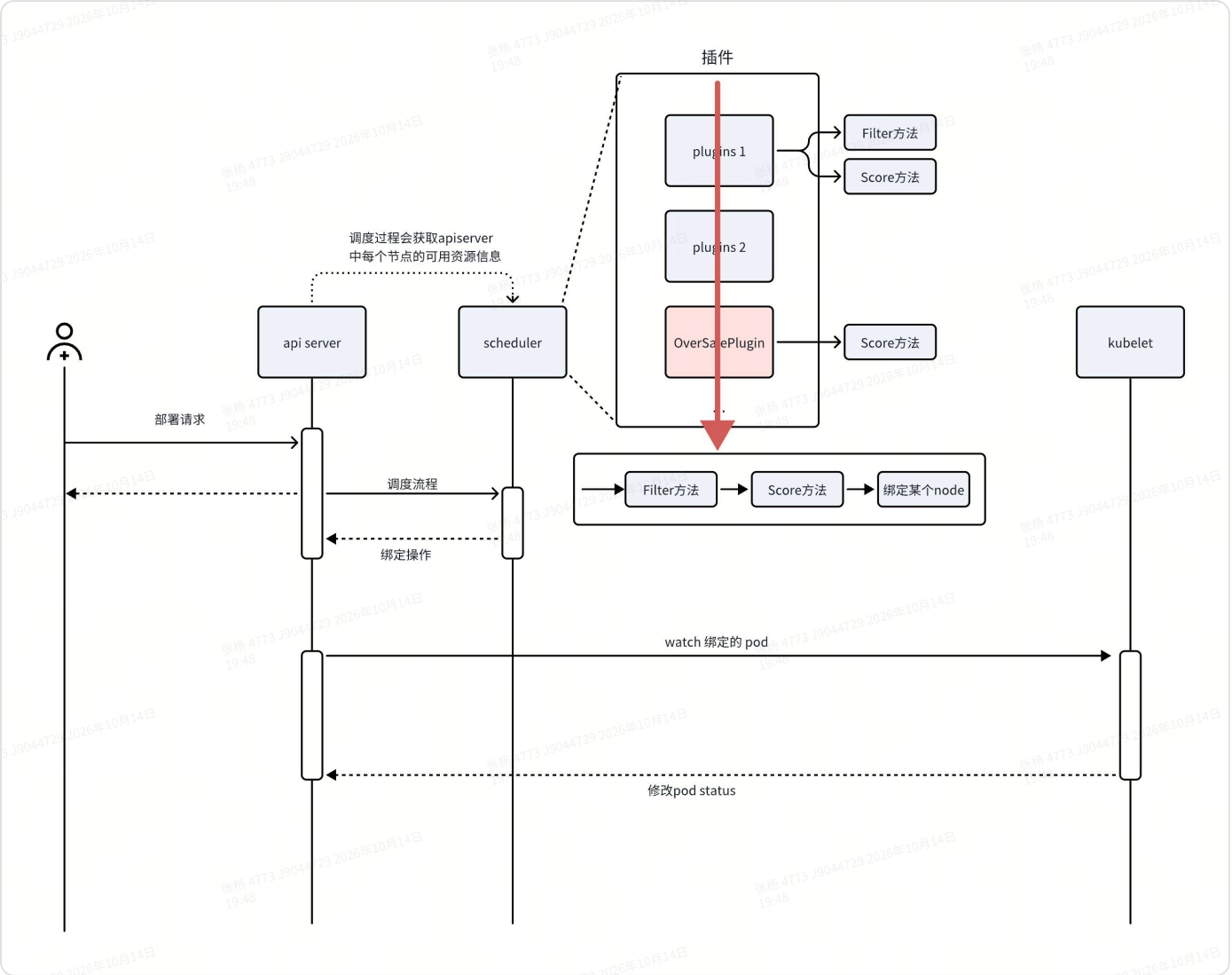
        if node == nil {
                err := fmt.Errorf("node is nil")
```

```
8                 klog.Error(err)
9                 return err
10        }
11
12        nodeAnnotations := node.Annotations
13        if nodeAnnotations == nil {
14                nodeAnnotations = make(map[string]string)
15        }
16
17        nodeAnnotations[consts.NodeAnnotationOriginalCapacityCPUKey] =
    node.Status.Capacity.Cpu().String()
18        nodeAnnotations[consts.NodeAnnotationOriginalAllocatableCPUKey] =
    node.Status.Allocatable.Cpu().String()
19
20        node.Annotations = nodeAnnotations
21
22        CPUOvercommitRatioValue, ok :=
    node.Annotations[consts.NodeAnnotationCPUOvercommitRatioKey]
23        if ok {
24                var newAllocatable, newCapacity *resource.Quantity
25                // get overcommit allocatable and capacity from annotation
    first
26                overcommitCapacity, ok :=
    node.Annotations[consts.NodeAnnotationOvercommitCapacityCPUKey]
27                if ok {
28                        quantity, err :=
    resource.ParseQuantity(overcommitCapacity)
29                        if err != nil {
30                                klog.Error(err)
31                        } else {
32                                newCapacity = &quantity
33                                klog.V(6).Infof("node %s cpu capacity by
    annotation: %v", node.Name, newCapacity.String())
34                        }
35                }
36
37                overcommitAllocatable, ok :=
    node.Annotations[consts.NodeAnnotationOvercommitAllocatableCPUKey]
38                if ok {
39                        quantity, err :=
    resource.ParseQuantity(overcommitAllocatable)
40                        if err != nil {
41                                klog.Error(err)
42                        } else {
43                                newAllocatable = &quantity
44                                klog.V(6).Infof("node %s cpu allocatable by
    annotation: %v", node.Name, newAllocatable.String())
```

```
45                          }
46                  }
47
48                  if newAllocatable == nil || newCapacity == nil {
49                          CPUOvercommitRatio, err :=
    cpuOvercommitRatioValidate(node.Annotations)
50                          if err != nil {
51                                  klog.Errorf("node %s %s validate fail,
    value: %s, err: %v", node.Name, consts.NodeAnnotationCPUOvercommitRatioKey,
    CPUOvercommitRatioValue, err)
52                          } else {
53                                  if CPUOvercommitRatio > 1.0 {
54                                          allocatable :=
    node.Status.Allocatable.Cpu()
55                                          capacity :=
    node.Status.Capacity.Cpu()
56                                          allocatableByOvercommit :=
    native.MultiplyResourceQuantity(core.ResourceCPU, *allocatable,
    CPUOvercommitRatio)
57                                          capacityByOvercommit :=
    native.MultiplyResourceQuantity(core.ResourceCPU, *capacity,
    CPUOvercommitRatio)
58                                          newAllocatable =
    &allocatableByOvercommit
59                                          newCapacity = &capacityByOvercommit
60
61                                          klog.V(6).Infof(
62                                                  "node %s %s capacity: %v,
    allocatable: %v, newCapacity: %v, newAllocatable: %v",
63                                                  node.Name, core.ResourceCPU,
64                                                  capacity.String(),
    newCapacity.String(),
65                                                  allocatable.String(),
    newAllocatable.String())
66                                  }
67                          }
68                  }
69
70                  if newAllocatable != nil && newCapacity != nil {
71                          node.Status.Allocatable[core.ResourceCPU] =
    *newAllocatable
72                          node.Status.Capacity[core.ResourceCPU] =
    *newCapacity
73                  }
74          }
75
76
```

```
77          return nil
78      }
```

schedulerPlugin执行流程



- 从当前node的annotation中获取oversale_cpu_ratio和cpu_usage_active
  - 如果oversale_cpu_ratio不存在或<=1，直接返回 100分
  - 如果cpu_usage_active不存在，直接返回 100分
  - 如果 cpu_usage_active 过期，直接返回 50分
  - 执行如下优选策略ScorePlugin：
    - 卡点一：调度前真实cpu使用率超过30%（cpu_usage_active > **0.3**），返回 0分
    - 卡点二：调度后cpu预估值 (cpu_usage_active*original_allcatable_cpu + pod.request.cpu) / node.Allocatable.cpu > **0.8**，返回 0分

      0.256_xxxxxx - > 25%

      pod request 20c

10c +

(25% * 40c + 20c) / 80c

- 阈值的默认值为0.3与0.8，支持以k8s env方式注入，便于在紧急情况下快速生效

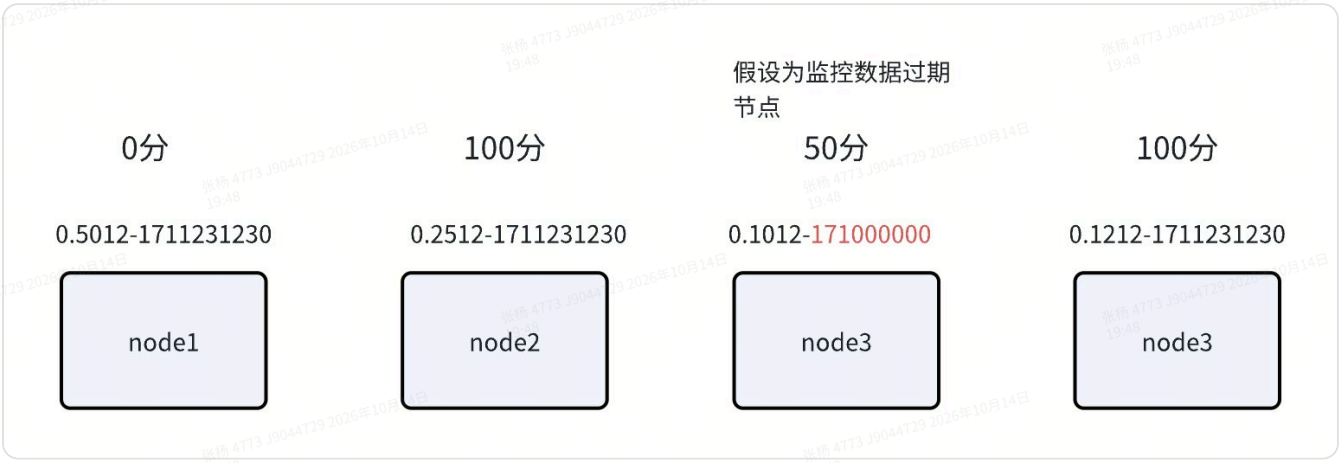
评分方法：使用 0 50 100 分数对不同情况节点进行打分：

插件权重：1

0分：代表是超卖节点，且触发卡点规则的节点

50分：代表是超卖节点，但监控数据过期，且符合卡点规则

100分：1. 代表符合超卖节点，并经过符合卡点规则 2. 没有监控标签 或 没有超卖标签的节点



代码块

```
// FilterPlugin is an interface for Filter plugins. These plugins are called at the
// filter extension point for filtering out hosts that cannot run a pod.
// This concept used to be called 'predicate' in the original scheduler.
// These plugins should return "Success", "Unschedulable" or "Error" in Status.code.
// However, the scheduler accepts other valid codes as well.
// Anything other than "Success" will lead to exclusion of the given host from
// running the pod.
type FilterPlugin interface {
    Plugin
    // Filter is called by the scheduling framework.
    // All FilterPlugins should return "Success" to declare that
    // the given node fits the pod. If Filter doesn't return "Success",
    // it will return "Unschedulable", "UnschedulableAndUnresolvable" or "Error".
    //
    // "Error" aborts pod scheduling and puts the pod into the backoff queue.
    //
```

```
    // For the node being evaluated, Filter plugins should look at the passed
    // nodeInfo reference for this particular node's information (e.g., pods
    // considered to be running on the node) instead of looking it up in the
    // NodeInfoSnapshot because we don't guarantee that they will be the same.
    // For example, during preemption, we may pass a copy of the original
    // nodeInfo object that has some pods removed from it to evaluate the
    // possibility of preempting them to schedule the target pod.
    Filter(ctx context.Context, state *CycleState, pod *v1.Pod, nodeInfo *NodeInfo) *Status
}
```

代码块

```
// PreScorePlugin is an interface for "PreScore" plugin. PreScore is an
// informational extension point. Plugins will be called with a list of nodes
// that passed the filtering phase. A plugin may use this data to update internal
// state or to generate logs/metrics.
type PreScorePlugin interface {
    Plugin
    // PreScore is called by the scheduling framework after a list of nodes
    // passed the filtering phase. All prescore plugins must return success or
    // the pod will be rejected
    // When it returns Skip status, other fields in status are just ignored,
    // and coupled Score plugin will be skipped in this scheduling cycle.
    PreScore(ctx context.Context, state *CycleState, pod *v1.Pod, nodes []*NodeInfo) *Status
}

// ScoreExtensions is an interface for Score extended functionality.
type ScoreExtensions interface {
    // NormalizeScore is called for all node scores produced by the same plugin's "Score"
    // method. A successful run of NormalizeScore will update the scores list and return
    // a success status.
    NormalizeScore(ctx context.Context, state *CycleState, p *v1.Pod, scores NodeScoreList) *Status
}

// ScorePlugin is an interface that must be implemented by "Score" plugins to rank
// nodes that passed the filtering phase.
```

```go
type ScorePlugin interface {
    Plugin
    // Score is called on each filtered node. It must return success and an integer
    // indicating the rank of the node. All scoring plugins must return success or
    // the pod will be rejected.
    Score(ctx context.Context, state *CycleState, p *v1.Pod, nodeName string) (int64, *Status)

    // ScoreExtensions returns a ScoreExtensions interface if it implements one, or nil if does not.
    ScoreExtensions() ScoreExtensions
}
```

- [x] ~~调度插件在 Filter 还是 Score 方法实现~~

区别：

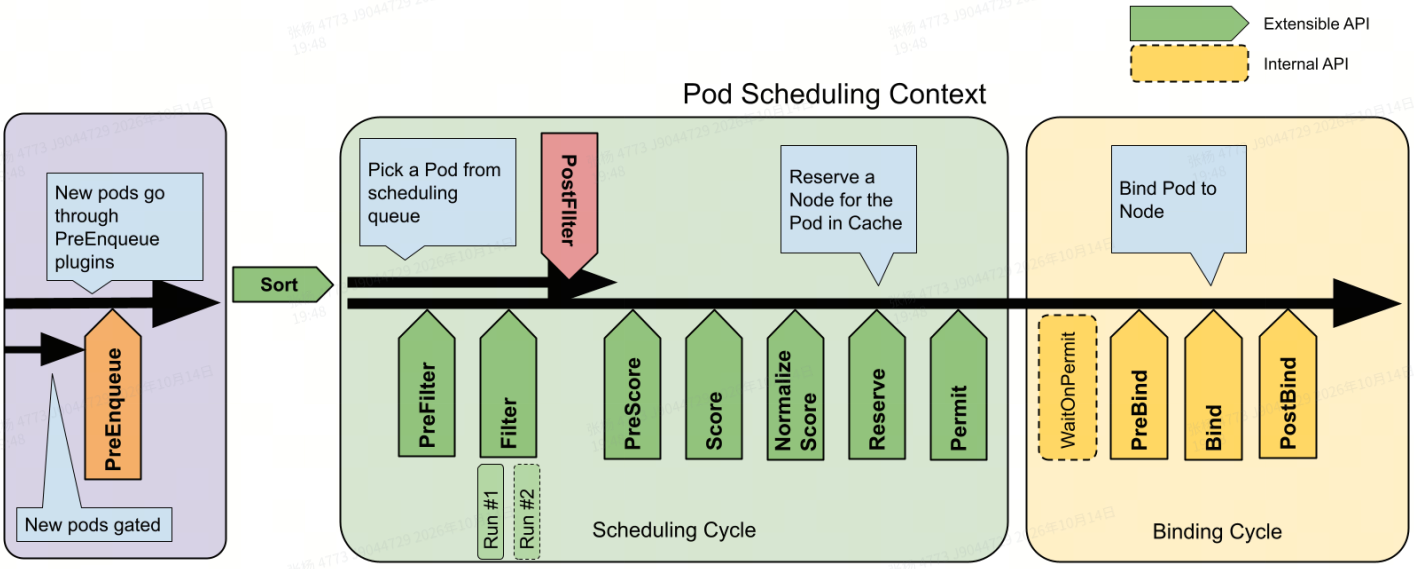
~~**Filter**: 这些插件用于过滤出不能运行该 Pod 的节点。对于每个节点，调度器将按照其配置顺序调用这些过滤插件。如果任何过滤插件将节点标记为不可行，则不会为该节点调用剩下的过滤插件。节点可以被同时进行评估。~~

Score: 这些插件用于执行“前置评分（pre-scoring）”工作，即生成一个可共享状态供 Score 插件使用。 如果 PreScore 插件返回错误，则调度周期将终止。

这些插件用于对通过过滤阶段的节点进行排序。调度器将为每个节点调用每个评分插件。 将有一个定义明确的整数范围，代表最小和最大分数。 在标准化评分阶段之后， 调度器将根据配置的插件权重合并所有插件的节点分数。

使用 Filter 类似强硬措施，如果不符合上述我们逻辑条件，就不会调度到这个节点。

使用 Score 类似软措施，如果不符合我们逻辑条件，只会降低这部份的评分，**总评分还是会按照所有插件总和获取**



https://kubernetes.io/zh-cn/docs/concepts/scheduling-eviction/scheduling-framework/#interfaces

5.1.3 功能权限设计

> 对齐产品业务需求，说明当前功能模块儿，涉及到的权限设计实现，包含：
>
> 1. 需要访问哪些服务（马上云服务按云安审要求会加入服务访问控制）
> 2. 业务访问权限控制方式，功能涉及权限矩阵的哪些角色、权限

5.1.4 性能需求分析

> *说明此功能的性能要求，包括访问量、处理时效、并发量等相关要求*

日均访问量	并发量	处理时效要求	性能指标要求	需要性能测试	流控降级支持

本功能无特殊要求。

5.1.5 业务表设计

> 说明本功能涉及的数据库变更设计，新增表或新增字段，新增索引；预估数据量级、是否考虑分表、是否需要热温冷数据归档处理等

修改 node_monitor pool_monitor cluster_monitor azone_monitor 表：

代码块

```
1   ALTER TABLE `node_monitor`
2           ADD COLUMN `cpu_limit` INT(10) NOT NULL DEFAULT '0' COMMENT 'cpu Limit
    字段 ';
3   ALTER TABLE `node_monitor`
4       ADD COLUMN `mem_limit` BIGINT(64) NOT NULL DEFAULT '0' COMMENT 'mem Limit字
    段';
5
6   ALTER TABLE `pool_monitor`
7           ADD COLUMN `cpu_limit` INT(10) NOT NULL DEFAULT '0' COMMENT 'cpu Limit
    字段 ';
8   ALTER TABLE `pool_monitor`
```

```
    ADD COLUMN `mem_limit` BIGINT(64) NOT NULL DEFAULT '0' COMMENT 'mem Limit字段';

ALTER TABLE `cluster_monitor`
        ADD COLUMN `cpu_limit` INT(10) NOT NULL DEFAULT '0' COMMENT 'cpu Limit字段 ';
ALTER TABLE `cluster_monitor`
    ADD COLUMN `mem_limit` BIGINT(64) NOT NULL DEFAULT '0' COMMENT 'mem Limit字段';

ALTER TABLE `azone_monitor`
        ADD COLUMN `cpu_limit` INT(10) NOT NULL DEFAULT '0' COMMENT 'cpu Limit字段 ';
ALTER TABLE `azone_monitor`
    ADD COLUMN `mem_limit` BIGINT(64) NOT NULL DEFAULT '0' COMMENT 'mem Limit字段';
```

建议考虑索引：

索引名：本功能不涉及 或 idx_created_by (created_by)

代码块

```
本功能不涉及 或 Alter table 语句
```

5.1.6 灵活配置设计(不涉及)

> *设计为了解决业务灵活性、系统稳定性、接口强耦等设计一些配置开关*

本功能无需考虑。

配置文件变更设计

序号	配置应用	配置key	配置value	类型
1	application.properties	-	-	修改
2	使用nacos配置中心 团体云应用接入Nacos指南			

业务数据字典变更设计

> 便于产品运维，直接调整数据字典即可实现业务扩展和变化

WAF模块暂无此功能

5.1.7 中间件使用设计(不涉及)

REDIS使用设计

> *说明此功能的redis使用点、key命名、存储数据大小预估、使用的必要性、过期机制介绍。*
>
> 针对可能的数据库访问热点，其他依赖服务访问并发瓶颈，评估并做缓存设计，redis
>
> 缓存需满足公司缓存设计规范，重点注意：
>
> 1. 不能依赖redis 做持久数据源，Redis 在主备切换下不能保证不丢数据
> 2. 所有key 必须有过期时间
> 3. 避免大key 设计（string类型控制在10KB以内，hash、list、set、zset元素个数不要超过5000。）

Key命名	数据大小	过期策略	使用场景	备注

ES使用设计

> 此功能使用到ES时，相应的索引设计（索引划分、容量），索引生命周期管理。 参考《ES 使用规范》

本功能不涉及。

消息总线使用设计

> *此功能作为生产者和消费者分别说明对应的要求，适用事件中心或直接使用Kafka。*

本功能不涉及。

平台类型	使用方式	topic名称/事件类型	规则名称	包含敏感信息	预估TPS	备注
消息总线/事件中心	生产者/消费者	WAF.gc-waf-portal:WAF:status.change	gcloudmw-cluster-change	否	1	

5.1.8 埋点要求分析/业务价值跟踪指标(不涉及)

> *分析说明此功能的埋点要求，后续埋点的使用简要描述：数据价值跟踪统计要求等。*
>
> *埋点类型：后端日志、前端埋点、事件链路上报、Prometheus指标等*

埋点一（xxxxxxx业务分析跟踪）

- 埋点类型： promethesus 指标/云监控指标/日志埋点
- 原因：为了统计xxxxx的调用量，计算xxxxx的价值
- 日志：逻辑执行至xxxxxx，均需要输出日志
- 格式："WAF[{wafId}]-{changeType}-调用日志安装/运维-{参数}:{响应结果}"

5.1.9 可靠性设计、业务连续性保障

> 当依赖的组件、外部系统或内部场景出现不可预知的异常时，保障本功能的可靠性的处理方式设计

风险点	依赖系统/应用/组件	不可用影响分析	处理方式	具备自动恢复能力
			熔断/降级/重试	具备

5.1.10 安全风险处理

> 此功能可能在线程安全、敏感信息泄漏、SQL注入、横向越权等方面存在风险，针对风险的规避处理设计
>
> 自动恢复能力设计，如外部接口down掉，我如何处理保障业务连续性
>
> 是否需要做渗透测试

本功能不涉及安全风险。

5.1.11 历史数据处理

> *说明此功能涉及的历史数据处理要求。对处理历史数据的方式、方法设计。*

本功能不涉及历史数据处理。

6. 业务监控设计

6.1 业务监控埋点

> 列举第5章节中所有功能的埋点监控要求，汇总告警配置接入清单。

监控项	监控方式	关键字	阀值	告警级别	告警方式
超售比打标失败	日志埋点	OverSaleLableFailed	1		

7. 数据库设计(不涉及)

> 本次详细设计所涉及的数据库设计全景方案

7.1 设计依据

> 说明数据被访问的频度和流量，最大数据存储量，数据增长量，存储时间等数据库设计依据。

1. xxxxx应用前管数据库：gcloud_xxx; xxxxx应用后管数据库：gcloud_xxx_manage
2. 不采用数据库数据库存储过程
3. 不使用数据库外键约束
4. 每个表必须有自增主键
5. 涉及到业务查询的字段需建立索引

7.2 数据库选型

> 对数据库选型进行确认。说明系统内应用的数据库种类、各自的特点，数据如何传递。

类型	数据库	版本	备注
关系型	MySQL	8.0.26	用于马上云常规业务数据存储
关系型	TiDB	6.5	用于马上云大规模业务数据存储
缓存	Redis	5.0.7	用于马上云分式锁、Session存储、高性能要求KV处理业务等，必须有过期时间
非关系型	ElasticSearch	7.4.2	用于云监控系统存储监控等数据

7.3 数据库结构

7.3.1 实体

> 本节列出所有核心的数据存储类的实体，实体间的关系，此节内容能对表的功能有个整体了解。列出所使用的库，实体对应的表。

7.3.2 表

> 表中每个数据项、记录和文件的标识、定义、长度及它们之间的相互关系。此节内容为数据库设计的主要部分。

8. 性能设计(不涉及)

8.1 性能要求

> 根据功能设计中分析的性能要求，设计性能保障设计，上线评审中解释性能测试报告，是否存在性能瓶颈。
>
> TODO: 从业务功能角度描述性能要求。

8.2 测试设计

> 设计压力测试的场景和要求，需要包括测试环境，测试拓扑，测试场景，测试数据量，是否支持水平扩展。
>
> 设计评审时强调给测试团队，测试用例评审时复核

8.3 流量控制

> 是否支持，支持了哪些控制。

9. 资源需求设计(不涉及)

> 明确本次设计相关资源（服务器、网络、数据库、中间件、Openstack 集群，K8S 集群等）诉求，邀请马上云SRE、基础运维、数据库中间件等资源交付团队人员参与评审。目的：达到资源交付团队提早了解资源诉求，提早准备
>
> 接口人： 数据库-吴洁； 中间件-张晓剑；服务器/k8s/openstack - 袁征

资源类别	使用场景描述	资源规格描述 （类别，规格大小，）	数目描述（实例数/集群数）	资源交付方确认人员	备注
服务器		物理机 40Core 256 G 5T	2台	马上云 SRE @xxx	

数据库		物理标准集群　40Core 128G 5T	1 套物理集群	DBA @xxx	
中间件-ES			1	中间件 SRE @xxx	
Openstack 集群		最小化集群。 计算节点数3		马上云 SRE @xxx	
K8S 集群		最小化集群。node		马上云 SRE @xxx	
Hbase 集群				大数据 SRE @xxx	
其他					

10. 容量增长评估(不涉及)

> 对本次设计相关业务功能，对已有中间件、数据库、缓存等组件是否需要扩容进行评估。

11. 其他设计

11.1 关键技术

> 关键技术指项目所采用的核心技术或技术难点、新技术(如果有的话)。
>
> 说明关键技术的名称、必要性、技术难点。

11.2 环境变量设计

> 如果该系统涉及租户侧系统环境变量变更，请在此详细列出，参考资料：马上云环境变量/应用启动参数使用规范。

序号	环境变量名称	环境变量作用	作用范围	使用时注意事项	

11.3 二开组件设计

> 如果该项目此次需求存在开源组件的二次开发，请在此提供二开设计文档。
>
> 参考模板：【模板】xxx功能二开设计文档。

计算资源池调度插件 功能二开设计文档

11.4 依赖计算资源非功能数据清理设计

> 对于各类产品依赖的Iaas 计算资源（ECS，ECI）内部产生的日志（不限于日志）等持久化数据，若根据业务使用会出现持续增长的，需设计对应的清理或归档机制。

11.5 历史资源兼容及升级设计

> 对于新功能迭代只对新产生的资源生效的，如何保证历史申请资源实例具备对应的功能。 设计评估新增功能是否能兼容历史资源实例，若不能兼容这里针对历史资源升级方式做相应设计。

11.6 其他设计

> 其他设计可放到此节。

内审结论

- **资源压缩**与**节点放大**方案**不并存**。
- **分阶段上线**
  - 上线先在马上云西永测试环境(sit)上线运行一段时间后，再上西永租户环境。（已经和sre与研发测达成一致）
  - dev7 2 sit 4 5 **西永 100**
- Scheduler plugins 代码仓库与容器组**一起放**，webhook 先放在资源池项目中

- Scheduler plugin 使用 **score** 实现

下一步行动

- [ ] 与业务方沟通 node status 字段变更后 需要跟 云监控 sre对齐
  - [x] ~~与龙军对齐，云监控目前不涉及~~
  - [x] ~~Sre 会用到这个字段 告警规则或是是否存在类似问题~~
- [x] ~~关于 webhook 性能压测这块需要做代办项 使用 client-go 按照设计好的qps进行~~
  - [x] ~~使用 client-go 测试 api-server 更新node status qps~~
  - [x] ~~搭配 webhook 测试 api-server 更新 node status qps~~
- [ ] 上线方案：需要与 sre 对齐切换方案 (资源压缩 -> 放大方案的切换)
  - [ ]
- [ ] 回滚方案开发时明确
- [ ] 当 webhook 停止后或超分率变小， 原本的超分节点是否会有影响？ **需要让测试在测试中覆盖此场景**

  例如： 20核 -> 超分2 变为 40核，运行一段时间后，webhook 停掉，api-server内信息会重新变为20核。
- [x] ~~cpu真实使用率不合适，需要改为最大值 已修改为~~
  - 近5分钟的**均值cpu_usage_avg_5m** = 100 - (avg by (instance) (avg_over_time(irate(node_cpu_seconds_total{mode="idle",instance="~~10.50.1.5:9100~~"}[1m])[5m:]))) * 100
  - 近1小时的**平均最大值cpu_usage_max_avg_1h** = 100 - (avg by (instance) (max_over_time(irate(node_cpu_seconds_total{mode="idle",instance="~~10.50.1.5:9100~~"}[5m])[1h:]))) * 100
  - 近1天的**平均最大值cpu_usage_max_avg_1d** =100 - (avg by (instance) (max_over_time(irate(node_cpu_seconds_total{mode="idle",instance="~~10.50.1.5:9100~~"}[5m])[1d:]))) * 100

    真实cpu利用率计算公式 = **0.2***cpu_usage_avg_5m + **0.3***cpu_usage_max_avg_1h + **0.5***cpu_usage_max_avg_1d （**权重可配置**）

研发中代办事项：

开发阶段：

- [x] ~~基础组件代码放置~~
  - [x] ~~Scheduler plugins https://codelab.msxf.com/gc-bs/kube-scheduler/-/tree/v0.24.9_lpf~~

- [x] ~~Webhook 先放在资源池代码仓库~~

- [x] ~~完成webhook kubebuilder 框架搭建~~
- [x] ~~已有资源池流程哪些需要修改~~

部署类：

- [ ] 部署在管理区 也就是控制节点上。 目前暂定，待开发中要部署的时候需要确认
- [ ] 部署流程(证书)需要再跟霞姐or奎哥讨论
    - [ ] 部署注册的时候要使用 vip 不能用真实物理ip

测试阶段：

- [ ] 测试获取node的qps
- [ ] 性能测试

上线类：

- [ ] 回滚方案 上线前需要确定
- [ ] 迁移方案：

排期 粗估

开发	-11/20
提侧时间	11/20-12/5 sit
稳定运行期	12/5-12/15 Sit stage
上线时间	12 月中旬

计画：

now- **11/20**左右：

开发 基础组件 webhook plugins 资源池服务开发。

11/20-12/15：

提侧并在sit中运行一段时间。

☐ 场景测试：

- 当 webhook 停止后或超分率变小， 原本的超分节点是否会有影响？ **需要让测试在测试中覆盖此场景**

  例如： 20核 -> 超分2 变为 40核，运行一段时间后，webhook 停掉，api-server内信息会重新变为20核。
- 当 webhook 崩溃，是否会影响 node节点上报
- 回滚场景：验证每个基础组件部署是失败后，快速回滚

☐ 性能测试：

Node 10s 15s 一次 100个节点

同步块 [该内容不支持导出查看]

12/1-12/15：

稳定运行期

12/15中旬上线

西永可用区k8s集群上线

替换方案：

暂定：

- 12月中上线后，新资源池优先使用节点放大方案。
- 存量资源池优先维持原方案，待业务量或使用率较低时，让SRE进行手动替换(调资源池超分比与移除原来limit request 设置标示)

Oversale 基础组件部署文档

张杨 4773 J9044729 2026年10月14日 19:48